한국성서대학교
KOREAN BIBLE UNIVERSITY

「2026년도 2학기 한국성서대학교 AI융합학부」

AIoT프로그래밍

입력 모듈 : 화재 감지, 자이로 센서
모터 실습 : 스테핑 모터, L298D DC 모터

7주차





목차

1. 화재감지 센서
2. 자이로 센서
3. 스테핑 모터
4. L298D DC 모터

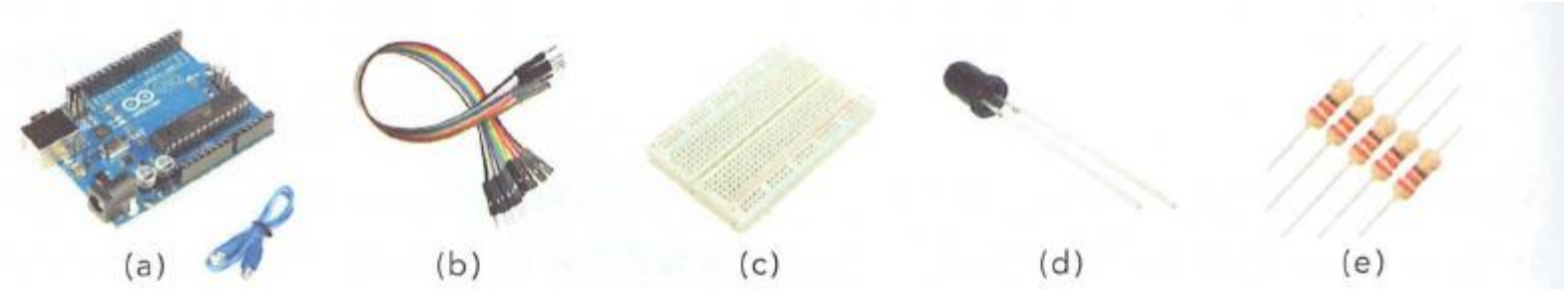
1. 화재 감지 센서

개요

- 아두이노를 활용하여 화재 감지 센서 제어에 대하여 알아봄
- 화재 감지 센서에서 출력되는 아날로그 전압을 디지털 값으로 AD 변환하여 아두이노 시리얼 모니터를 통하여 확인함.

사용 디바이스

- 아두이노 우노 보드(a), 점퍼 케이블(b), 브레드 보드(c), 화재감지센서(d), 저항_10k(e)



1. 화재 감지 센서

- 화재감지센서(포토트랜지스터)
  - 화재 감지 센서 또는 불꽃 감지 센서라고 불리며 적외선 및 빛을 감지하여 전기에너지로 변환 시켜 증폭하는 작용을 하는 센서
  - 다리가 긴 쪽 : 에미터(Emitter)
  - 다리가 짧은 쪽 : 컬렉터(Collector)
  - 센서는 적외선(IR) 스펙트럼에서 발생하는 불꽃의 빛을 판별함. 주로 760nm~1100nm 범위의 적외선을 감지할 수 있음. 또한 이산화탄소발생 구간에서 적외선이 방출되는 에너지가 커지는 특성을 이용하여 화재 여부를 감지함.

CO_2는 특정 적외선 파장(보통 **4.26μm** 근처)을 흡수하거나 방출하는 특성이 있음

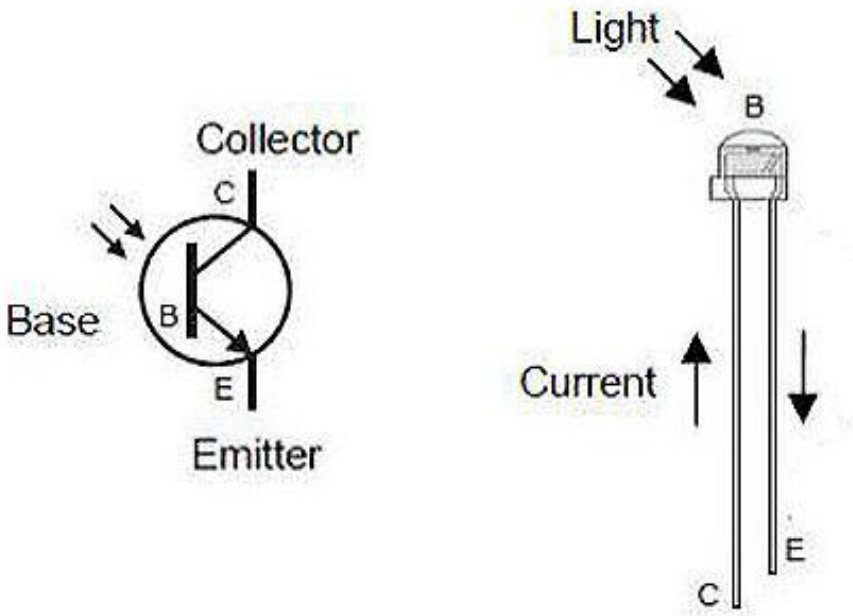
불꽃은 여러 파장의 빛을 동시에 냄
불이 타면 눈에 보이는 **가시광선(빨강, 노랑)** 만 나오는 게 아니라 **적외선(IR), 자외선(UV)** 같은 보이지 않는 파장도 함께 나옴



1. 화재 감지 센서

- 화재감지센서(포토트랜지스터)
  - 다리가 긴 쪽 : 에미터(Emitter)
  - 다리가 짧은 쪽 : 컬렉터(Collector)

구분	기능
Collector(컬렉터)	들어온 적외선을 감지 → 전류 변화 발생
Emitter(에미터)	전류가 빠져나가는 경로, 감지된 전류를 출력



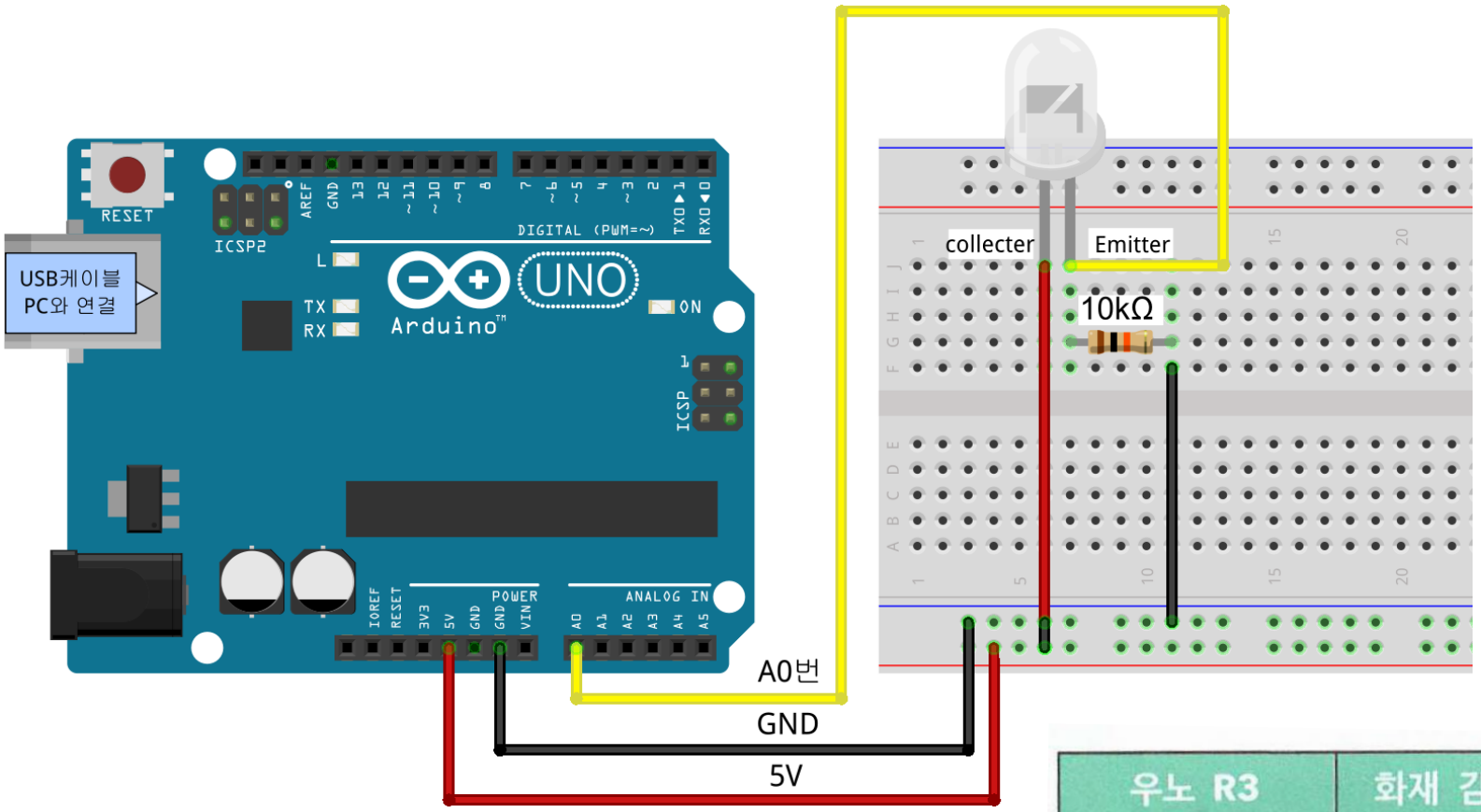
1. 화재 감지 센서

- 화재 감지 센서 – 감지 각도
  - 화재 감지 센서의 감지 각도는 아래 그림과 같이 일반적으로 60도이며 이 범위에 해당하는 각도에서만 불꽃 감지가 가능함.



  - 센서의 감지 범위를 넓히고 효과적으로 커버하기 위해서 화재 감지 센서를 높은 위치에 설치하거나 각 센서가 서로 겹치는 범위를 가지도록 배치하여 사각지대를 최소화 해야 함.

2. 화재 감지 센서 - 회로도



우노 R3	화재 감지 센서
아날로그 0번핀	에미터 핀
5V핀	브레드보드 5V
GND핀	브레드보드 GND

3. 화재 감지 센서 – 실습 코드

```
#define flame A0 // 센서 핀 A0로 설정
int val ;
void setup() {  // 초기화
  Serial.begin(9600); // 시리얼통신속도 설정
  pinMode(flame, INPUT);  // 센서 핀 입력 핀 설정
}
void loop() { // 무한루프
  val = analogRead(flame);  // 센서의 아날로그 값을 읽어와 변수에 저장
  Serial.println(val);  // 변수 값을 시리얼모니터에 출력
  delay(200); // 딜레이 200 ms
}
```

3. 화재 감지 센서 – 실습 코드

- 센서 출력 = 전압(아날로그 신호)
  - 포토트랜지스터가 적외선을 감지하면

    → 전류가 변하고

    → 그 전류가 전압 변화로 바뀌어서 A0 핀에 전달
- 예 :
  - 적외선 강함 → 전압 높음
  - 적외선 약함 → 전압 낮음

- analogRead()는 그 전압을 측정 → 숫자로 변환
  - 아두이노의 ADC(Analog to Digital Converter)가 0~5V 전압을

    0~1023 사이의 정수로 변환

4. 화재 감지 센서 – 실습 참고 영상



5. 자이로 센서

- 개요
  - 아두이노를 활용한 자이로 센서 모듈 제어에 대하여 알아봄
  - 자이로 센서 모듈과 우노 간의 I2C 통신을 통하여 자이로와 기울기 X, Y, Z 축 값을 얻은 후 아두이노 스케치 시리얼 모니터를 통해 확인함.

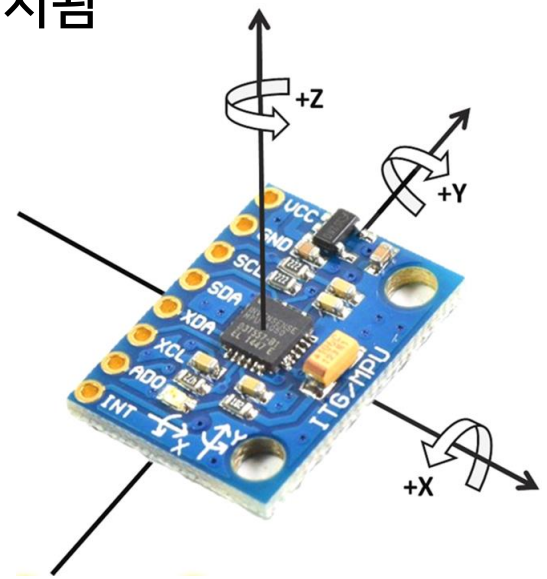
- 사용 디바이스
  - 아두이노 우노 보드, 점퍼 케이블, 자이로 센서 모듈

5. 자이로 센서

- MPU6050 이란?
  - MPU6050은 6축(6DOF, Degrees of Freedom) 모션 트래킹 장치로, 3축 자이로스코프와 3축 가속도계를 하나의 칩에 결합한 센서임.
  - 이 센서는 주로 드론, 로봇, 게임 장비, 웨어러블 디바이스 등에서 자세 추적, 모션 감지, 관성 측정 등 다양한 용도로 사용됨.
  - 3축 자이로스코프 및 가속도계 등의 주요 기능을 가짐
  - 3축 자이로스코프 : 각속도(회전 속도)를 측정하며, 각 축(X, Y, Z)의 회전율을 감지함. 단위는 도/초(degrees per second, dps)
  - 3축 가속도는 가속도를 측정하여 각 축(X, Y, Z)에서의 가속도를 감지함. 단위는 g(중력 가속도)임.

5. 자이로 센서 – 원리

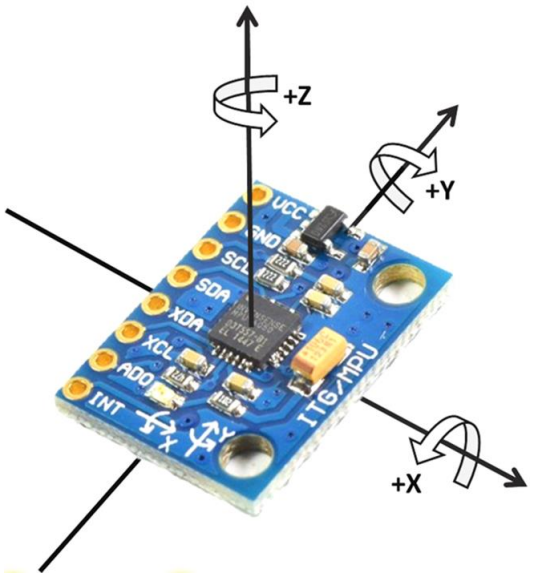
- 3축 가속도계(Accelerometer)의 원리
  - 동작 원리: MEMS 질량-스프링 구조
  - 가속도계는 내부에 작은 진자(질량, Mass)와 스프링 구조가 있는 MEMS(Micro Electro Mechanical System) 로 이루어져 있음
  - 센서가 움직이거나 기울어지면→ 내부의 미세 질량이 관성 때문에 움직임
  - 그 이동 거리가 정전용량(Capacitance) 변화로 감지됨
  - 정전용량 변화를 전기 신호로 변환
  - X, Y, Z 축 각각의 가속도를 계산하여 기울어져 있는 정도를 나타냄



5. 자이로 센서 – 원리

- 3축 자이로스코프(Gyroscope)의 원리
  - 동작 원리: 진동하는 MEMS 구조의 Coriolis Effect(코리올리 힘)
  - MPU6050의 자이로는 진동형 MEMS 자이로임.
  - 내부에 아주 작은 구조물이 일정 방향으로 심하게 진동하고 있음
  - 센서가 회전하면, 회전하는 물체의 진동 방향이 변하며 코리올리 힘이 발생
  - 이 힘이 진동 방향을 약간 변화시키고
  - 그 변화량을 정전용량 혹은 저항 변화로 감지
  - X, Y, Z 각 축에 대한 각속도(°/s) 측정

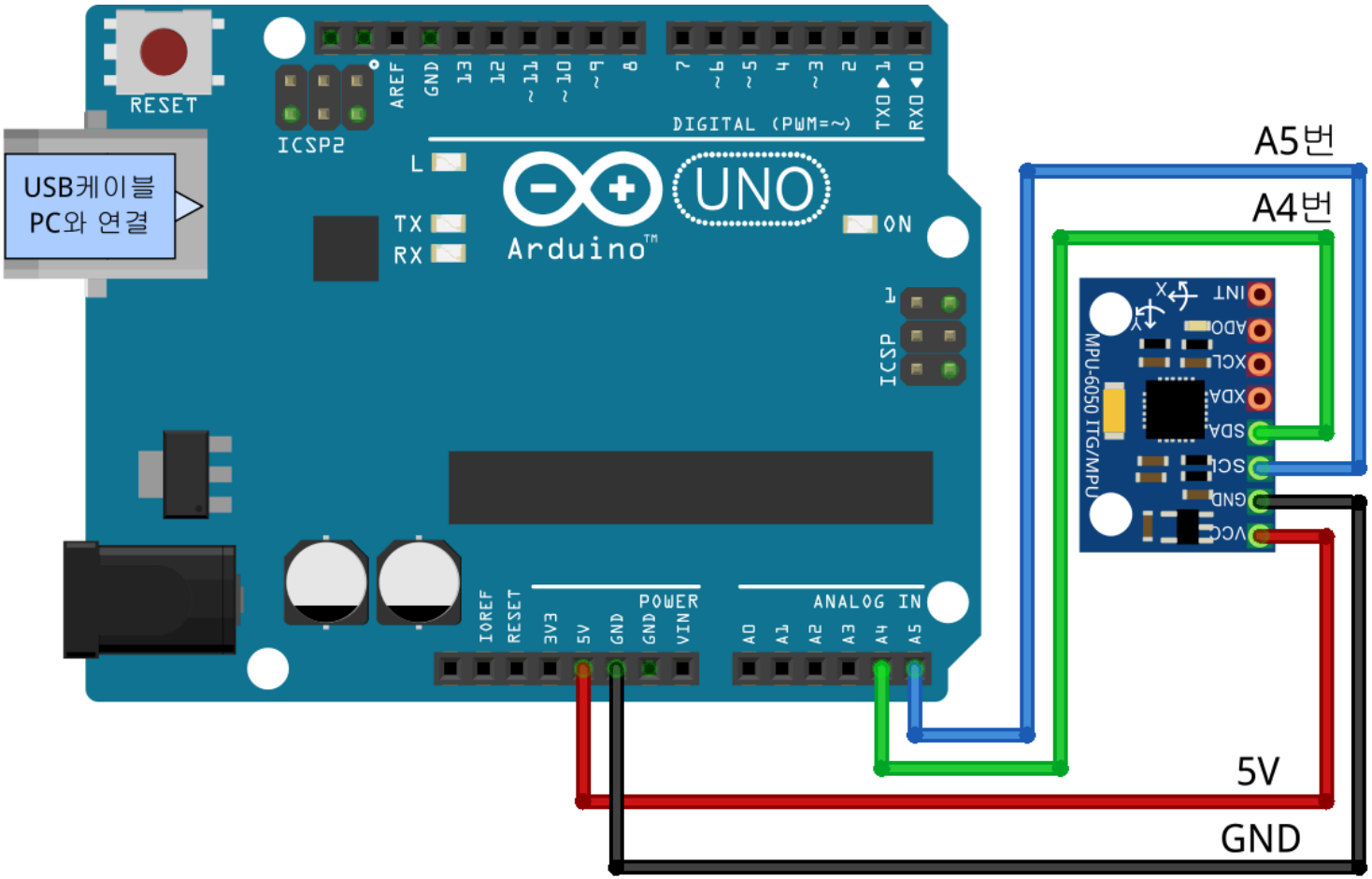
(각속도: 물체가 얼마나 빨리 회전하는지 나타내는 값)



5. 자이로 센서 – 통신

- I2C 통신이란?
  - I2C(Inter-Integrated Circuit)는 SDA(Data) : 데이터 송수신, SCL(Clock) : 클록 신호, 두 선을 이용해 여러 장치와 동시에 통신하는 버스 방식
- 핵심 특징
  - 선 두 개만 사용
  - 여러 장치를 하나의 버스에 연결 가능
  - 각 장치는 고유 주소(I2C Address) 가 있음
  - 마스터(보통 아두이노)와 슬레이브(센서) 구조

5. 자이로 센서 – 회로도


RESET
ICSP2
AREF
GND
13
12
~11
~10
~9
8
7
~6
~5
4
~3
2
TX→1
RX←0
DIGITAL (PWM=~)
L
TX
RX
Arduino™
UNO
ON
ICSP
USB케이블
PC와 연결
POWER
IOREF
RESET
3V3
5V
GND
GND
VIN
ANALOG IN
A0
A1
A2
A3
A4
A5
A5번
A4번
INT
ADO
XCL
XDA
SDA
SCL
GND
VCC
MPU-6050 ITG/MPU
5V
GND

5. 자이로 센서 – 실습 코드

```
/* I2C 통신을 위해 Wire.h 헤더 파일을 include 한다. */
#include <Wire.h>

// MPU6050의 I2C 주소(기본값은 0x68)
// ADO 핀을 HIGH로 하면 주소는 0x69가 됨
const int MPU_addr = 0x68;


// MPU6050 센서로부터 읽을 16비트 데이터 변수 선언
int16_t AcX, AcY, AcZ;  // 가속도 센서 값 (X, Y, Z축)
int16_t Tmp;            // 온도 센서 값
int16_t GyX, GyY, GyZ;  // 자이로 센서 값 (X, Y, Z축)


void setup() // 초기화 함수 (아두이노 전원이 켜지면 한 번 실행됨)
{
  Wire.begin(); // I2C 통신 시작 (SDA/SCL 핀 활성화)

  // MPU6050을 깨우기 위한 설정
  Wire.beginTransmission(MPU_addr); // MPU6050에 데이터 전송 시작
  Wire.write(0x6B);                 // PWR_MGMT_1 레지스터 선택 : 전원관리와 절전모드 관리
  Wire.write(0);                    // 0을 쓰면 sleep 모드 해제 → 센서 활성화
  Wire.endTransmission(true);       // 전송 끝

  Serial.begin(9600); // 시리얼 통신 초기화 (PC와 데이터 주고받기)
}
```

5. 자이로 센서 – 실습 코드

```
/*
 MPU6050 모듈은 총 6축(3축 가속도 + 3축 자이로)을 측정할 수 있으며
 이 데이터를 이용해 기울기, 회전 등 다양한 동작을 해석할 수 있다.
 PC의 시리얼 모니터에서 값을 출력하여 확인할 수 있다.
*/
void loop() // 무한 반복 (센서 데이터를 계속 읽음)
{
  // --------------- MPU6050의 레지스터 처음 위치 설정 ---------------
  Wire.beginTransmission(MPU_addr);
  Wire.write(0x3B);  // ACCEL_XOUT_H 레지스터 주소 (가속도 X축 상위 바이트가 저장된 주소)
  Wire.endTransmission(false);
  // false → 반복 시작(restart condition), 연속 읽기 준비

  // --------------- 총 14바이트 요청 ---------------
  // ACCEL X(2) + Y(2) + Z(2) + TEMP(2) + GYRO X(2) + Y(2) + Z(2)
  Wire.requestFrom(MPU_addr, 14, true);
// MPU6050은 하나의 16비트 데이터를 상위 8비트와
하위 8비트로 나누어 보냄
  // ----------- 가속도 데이터 읽기 -----------
  AcX = Wire.read() << 8 | Wire.read();  // X축
  AcY = Wire.read() << 8 | Wire.read();  // Y축
  AcZ = Wire.read() << 8 | Wire.read();  // Z축

  // ----------- 온도 데이터 읽기 -----------
  Tmp = Wire.read() << 8 | Wire.read();
```

바이트 위치	내용	설명
0-1	ACCEL_X	X축 가속도 (2바이트)
2-3	ACCEL_Y	Y축 가속도 (2바이트)
4-5	ACCEL_Z	Z축 가속도 (2바이트)
6-7	TEMP	온도 (2바이트)
8-9	GYRO_X	X축 각속도 (2바이트)
10-11	GYRO_Y	Y축 각속도 (2바이트)
12-13	GYRO_Z	Z축 각속도 (2바이트)

5. 자이로 센서 – 실습 코드

```
// ----------- 자이로 데이터 읽기 -----------
  GyX = Wire.read() << 8 | Wire.read();  // X축 회전속도
  GyY = Wire.read() << 8 | Wire.read();  // Y축 회전속도
  GyZ = Wire.read() << 8 | Wire.read();  // Z축 회전속도


  // --------------- 시리얼 출력 ---------------
  Serial.println(F("MPU-6050"));

  Serial.print(F("accel x,y,z: "));
  Serial.print("AcX = "); Serial.print(AcX);
  Serial.print(" | AcY = "); Serial.print(AcY);
  Serial.print(" | AcZ = "); Serial.print(AcZ);

  Serial.print(F("temperature: "));
  Serial.print(" | Tmp = ");
  Serial.print(Tmp / 340.00 + 36.53);
  // 데이터시트에 있는 온도 변환 공식 적용

  Serial.print(F("gyro x,y,z: "));
  Serial.print(" | GyX = "); Serial.print(GyX);
  Serial.print(" | GyY = "); Serial.print(GyY);
  Serial.print(" | GyZ = "); Serial.println(GyZ);

  delay(333); // 0.33초마다 한 번씩 데이터를 출력
}
```

센서 로우 데이터

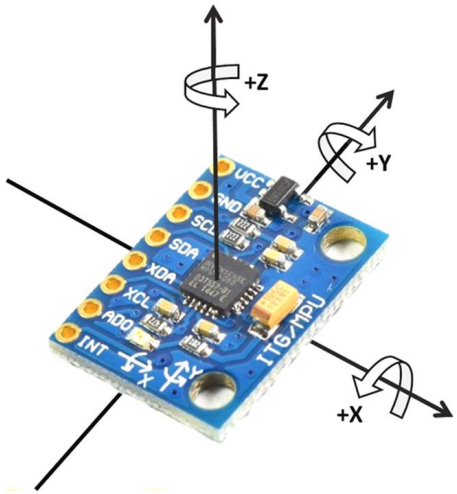
```
Gyro x,y,z :  X = 157 | Y = 526 | Z = 522 Accel x,y,z: X = 648 | Y = 16328 | Z = -1088 temperature: 30.08
Gyro x,y,z :  X = -421 | Y = -265 | Z = -73 Accel x,y,z: X = 712 | Y = 15984 | Z = -1176 temperature: 30.13
Gyro x,y,z :  X = -354 | Y = 357 | Z = 41 Accel x,y,z: X = 588 | Y = 16232 | Z = -776 temperature: 30.13
Gyro x,y,z :  X = -350 | Y = 168 | Z = -231 Accel x,y,z: X = 840 | Y = 15860 | Z = -1080 temperature: 30.13
Gyro x,y,z :  X = -359 | Y = 380 | Z = -272 Accel x,y,z: X = 708 | Y = 16120 | Z = -1064 temperature: 30.13
Gyro x,y,z :  X = -537 | Y = -9 | Z = -252 Accel x,y,z: X = 812 | Y = 16152 | Z = -1064 temperature: 30.22
Gyro x,y,z :  X = -368 | Y = -45 | Z = 33 Accel x,y,z: X = 720 | Y = 15996 | Z = -1080 temperature: 30.13
Gyro x,y,z :  X = -2070 | Y = 495 | Z = -624 Accel x,y,z: X = 804 | Y = 16464 | Z = -1464 temperature: 30.22
Gyro x,y,z :  X = -10315 | Y = -1608 | Z = -4464 Accel x,y,z: X = -240 | Y = 16480 | Z = 748 temperature: 30.32
Gyro x,y,z :  X = -16229 | Y = -539 | Z = -7701 Accel x,y,z: X = -832 | Y = 16368 | Z = 2964 temperature: 30.13
Gyro x,y,z :  X = -11727 | Y = -2215 | Z = 1546 Accel x,y,z: X = -2556 | Y = 12708 | Z = 9812 temperature: 30.27
Gyro x,y,z :  X = -14777 | Y = -6738 | Z = 2760 Accel x,y,z: X = -1348 | Y = 11716 | Z = 11980 temperature: 30.18
Gyro x,y,z :  X = -4802 | Y = -3595 | Z = 954 Accel x,y,z: X = -1420 | Y = 8724 | Z = 13976 temperature: 30.22
Gyro x,y,z :  X = -8448 | Y = -4116 | Z = 718 Accel x,y,z: X = -440 | Y = 6156 | Z = 15132 temperature: 30.22
Gyro x,y,z :  X = -5705 | Y = -1601 | Z = 2407 Accel x,y,z: X = 144 | Y = 5044 | Z = 16480 temperature: 30.18
Gyro x,y,z :  X = -1581 | Y = -230 | Z = -3580 Accel x,y,z: X = 228 | Y = 2204 | Z = 16964 temperature: 30.32
Gyro x,y,z :  X = -1874 | Y = -5215 | Z = -1929 Accel x,y,z: X = 1060 | Y = 1664 | Z = 17196 temperature: 30.22
Gyro x,y,z :  X = -12329 | Y = -914 | Z = 4275 Accel x,y,z: X = 1692 | Y = 380 | Z = 17860 temperature: 30.32
Gyro x,y,z :  X = 278 | Y = 2372 | Z = -16 Accel x,y,z: X = 2244 | Y = -352 | Z = 17096 temperature: 30.22
Gyro x,y,z :  X = 1361 | Y = 6882 | Z = -6603 Accel x,y,z: X = 1076 | Y = 768 | Z = 16348 temperature: 30.18
Gyro x,y,z :  X = 1425 | Y = 2317 | Z = -1294 Accel x,y,z: X = 1256 | Y = 704 | Z = 17100 temperature: 30.27
Gyro x,y,z :  X = -986 | Y = -5755 | Z = 5637 Accel x,y,z: X = 1012 | Y = 652 | Z = 17220 temperature: 30.32
Gyro x,y,z :  X = 1062 | Y = 104 | Z = -391 Accel x,y,z: X = 1104 | Y = 1672 | Z = 17180 temperature: 30.27
Gyro x,y,z :  X = 3419 | Y = -2418 | Z = 5645 Accel x,y,z: X = 928 | Y = 2220 | Z = 16448 temperature: 30.27
Gyro x,y,z :  X = 5108 | Y = -1476 | Z = -2156 Accel x,y,z: X = 776 | Y = 2680 | Z = 17724 temperature: 30.32
Gyro x,y,z :  X = -1212 | Y = 410 | Z = -1691 Accel x,y,z: X = 1056 | Y = 2336 | Z = 17040 temperature: 30.37
Gyro x,y,z :  X = -883 | Y = -459 | Z = -806 Accel x,y,z: X = 1060 | Y = 2604 | Z = 17416 temperature: 30.32
Gyro x,y,z :  X = -2987 | Y = -1823 | Z = -434 Accel x,y,z: X = 1340 | Y = 1724 | Z = 15784 temperature: 30.32
Gyro x,y,z :  X = -6591 | Y = 12 | Z = 778 Accel x,y,z: X = 1464 | Y = 1784 | Z = 17900 temperature: 30.46
Gyro x,y,z :  X = -5191 | Y = 13 | Z = 834 Accel x,y,z: X = 1784 | Y = -1400 | Z = 16368 temperature: 30.27
Gyro x,y,z :  X = -15233 | Y = -11148 | Z = 4250 Accel x,y,z: X = 1388 | Y = -3032 | Z = 17676 temperature: 30.32
Gyro x,y,z :  X = -7297 | Y = 11439 | Z = 5311 Accel x,y,z: X = 3880 | Y = -9036 | Z = 13184 temperature: 30.37
Gyro x,y,z :  X = -4737 | Y = 3346 | Z = 932 Accel x,y,z: X = 1836 | Y = -10548 | Z = 13308 temperature: 30.37
Gyro x,y,z :  X = -10874 | Y = 3201 | Z = -2876 Accel x,y,z: X = 508 | Y = -12864 | Z = 11048 temperature: 30.41
Gyro x,y,z :  X = -4214 | Y = 593 | Z = -142 Accel x,y,z: X = 1372 | Y = -15272 | Z = 7252 tempe
```

로우 데이터로부터 가속도 계산

- MPU6050의 가속도 센서(accelerometer) 는 물체가 받는 가속도를 측정하는데, 그 기준 단위가 g(gravity)임.
- 예) 2바이트는 -32768 ~ 32768범위 데이터
  - 아무 움직임 없이 가만히 둘 때
    → Z축에서 약 +1g(실제 중력 값)
  - 기울이면 X축이나 Y축에서 g 값이 달라짐
  - 기기를 평평하게 놓으면
  - Z축 +1g, X,Y 0g
  - X축을 90° 기울이면
  - X축 +1g, Z축 0g

$$\text{가속도 (g)} = \frac{\text{Raw Value (AcX, AcY, AcZ)}}{\text{Sensitivity Scale Factor}}$$

범위	Scale Factor (LSB/g)
±2g	16384
±4g	8192
±8g	4096
±16g	2048



로우 데이터로부터 자이로스코프 계산

- MPU6050 같은 자이로스코프 센서는 각속도(angular velocity)를 측정함. 각속도란, 단위 시간당 회전하는 속도이고 단위는 보통 °/s (degrees per second, 초당 회전 각도)로 나타냄.
- ±250°/s 기준일 때
  - ±32768 raw → ±250°/s 기준 ➔ 아래 공식으로 계산할 수 있음.

$$\text{deg/s} = \text{raw} \times \frac{250}{32768}$$

범위	Scale Factor (LSB/°/s)
±250°/s	131
±500°/s	65.5
±1000°/s	32.8
±2000°/s	16.4

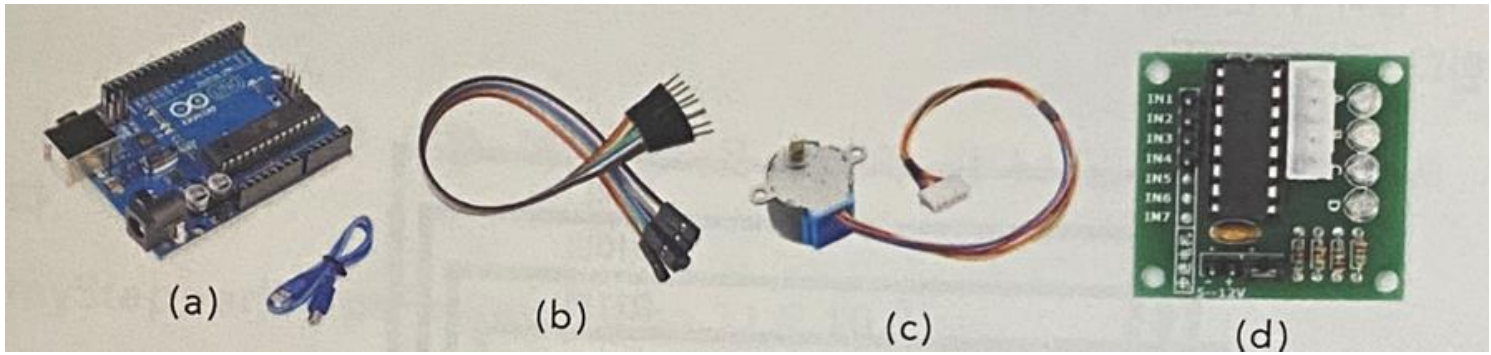
3. 스테핑 모터

개요

- 아두이노를 활용하여 스테핑 모터 제어에 대하여 알아봄
- 우노에서 스테핑 모터로 펄스를 보내 0.5초마다 정방향, 역방향으로 변화시켜 구동하는 동작을 구현

사용 디바이스

- 아두이노 우노 보드(a), 점퍼 케이블(b), 스테핑 모터(c), 모터드라이버(d)

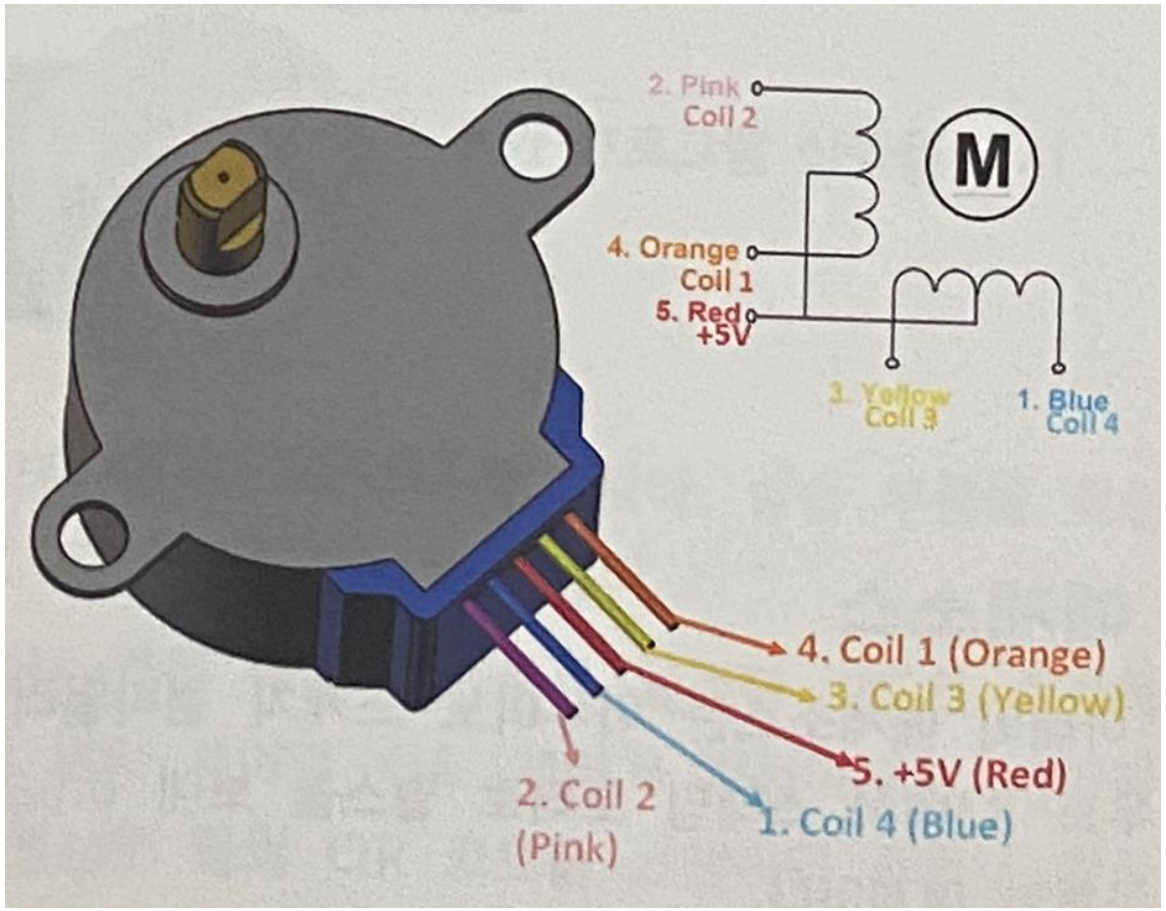


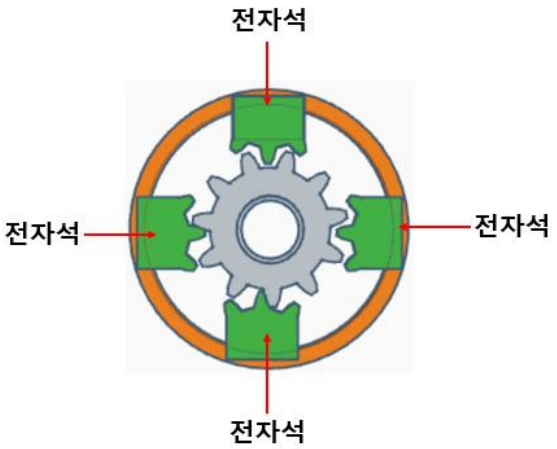
3. 스테핑 모터

- 스테핑 모터 배경 지식
  - 스테핑 모터는 전원을 입력하고 펄스를 넣어줘야만 회전하는 모터
  - 또한 <그림 1>처럼 구조를 띄고 있으며, 전원핀과 펄스 입력핀을 확인해 배선할 수 있음.
  - 특히 스테핑 모터 구동을 위해서는 1, 2, 3, 4번핀에 펄스를 입력해야 함. 보통 스테핑 모터축은 1펄스당 1.8도의 회전을 하는데 이 수치는 스테핑 모터마다 다를 수 있음(실습 예제는 28BYJ-48 모터: 한 펄스당 회전 각도는 0.1758°)
  - 펄스 단위로 움직이는 모터이므로 정확한 각도, 속도 제어가 가능함. (예를 들어 90도를 움직인다고 했을 때 펄스를 50개만 넣어 주면 됨)
  - 속도 또한 1분에 30바퀴를 회전한다고 했을 때 1분당 6000펄스 초당 100펄스를 넣어 준다면 1분에 30바퀴를 정확히 회전할 수 있음.

3. 스테핑 모터

- 스테핑 모터 구조





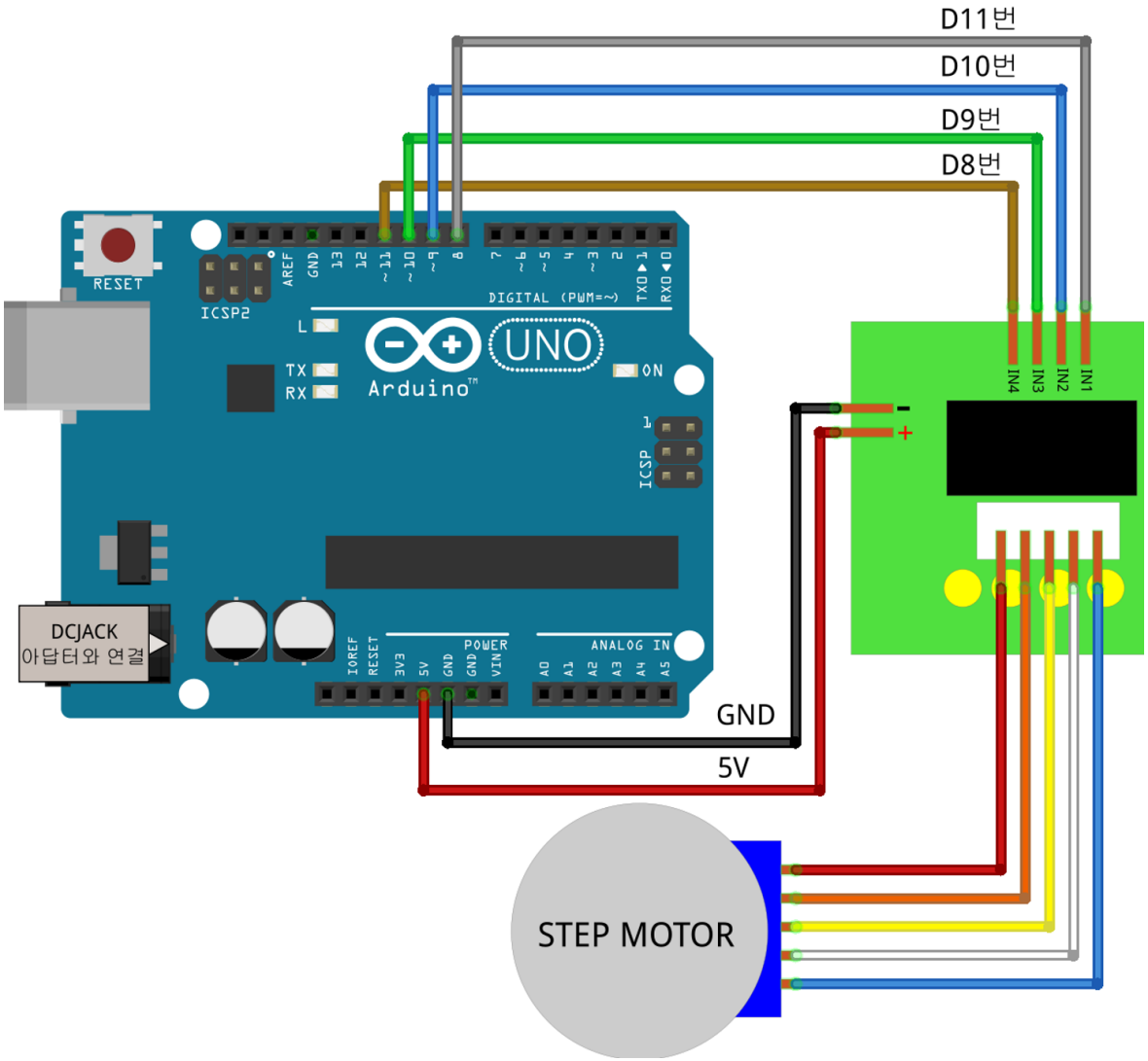
<그림 1. 스테핑 모터 구조>

3. 스테핑 모터

- 스테핑 모터 구현 소스 코드 및 구현
  - 우노 R3 디지털 8번핀을 모터 드라이버의 IN1에 연결하고 디지털 9번핀을 모터 드라이버 IN2핀에 연결
  - 우노 R3 디지털 10번핀과 디지털 11번핀 또한 모터 드라이버 IN3핀과, IN4핀에 각각 연결
  - 우노 R3의 5V핀은 모터 드라이버 5V핀에 우노 R3의 GND핀은 모터 드라이버 GND핀에 연결하고 스테핑 모터는 모터 드라이버의 커넥터에 알맞게 꽂으면 됨. 또한 모터 구동 시 우노 R3의 USB 전원만으로는 무리가 될 수 있으니 배터리 케이스나 DC 전원 아답터 등을 사용하는 것을 권장함.
  - 모터 드라이버의 5V핀과 GND핀에 전원을 투입할 때 반대로 전원을 투입하지 않도록 주의. 5V, GND핀에 전원을 입력할 때는 반드시 극성에 유의해 연결함.

3. 스테핑 모터

- 스테핑 모터 구현 회로도



3. 스테핑 모터 – 실습 코드

```
#include <Stepper.h>  // 스테핑 모터 라이브러리
// 상수 정의
const int stepsPerRevolution = 2048; //스테핑 모터가 한 바퀴(360°)를 돌기 위해 필요한 스텝 수
// 2048: 한 바퀴(360도), 1024: 반바퀴(180도)
// 핀 연결 (IN4, IN2, IN3, IN1 순서)
Stepper myStepper(stepsPerRevolution, 11, 9, 10, 8);//myStepper 객체 생성, 핀 설정
void setup() {
  // 모터 속도 설정 (rpm)
  myStepper.setSpeed(14);//모터 속도를 14 rpm(분당 회전 수)으로 설정
}
void loop() {
  // 정방향 회전: 시계 반대 방향
  myStepper.step(stepsPerRevolution); //step() 매개변수 > 0 : 정방향
  delay(500);  // 500ms 지연
  // 역방향 회전: 시계 방향
  myStepper.step(-stepsPerRevolution); //매개변수 < 0 : 역방향
  delay(500);  // 500ms 지연
}
```

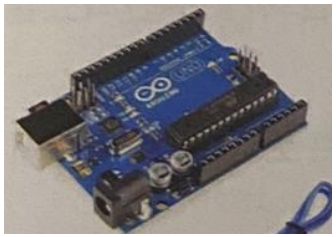
3. 스테핑 모터 – 실습 코드

- Stepper myStepper (steps Per Revolution, 11,9,10,8) : 8,10,9,11 숫자들은 스테핑 모터의 4개의 제어 핀을 아두이노 보드의 디지털 핀 번호에 연결하기 위한 것임.
- StepsPerRevolution : 모터의 회전당 스텝 수를 나타내는 변수임. 예를 들어 스테핑 모터가 한바퀴(360도) 회전하기 위해 몇 개의 스텝을 실행해야 하는지를 나타냄. 이 값은 모터의 해상도에 따라 다르며, 일반적으로 1024, 2048 값 등으로 설정
- myStepper.setSpeed(value) : 아두이노 함수에서 스테핑 모터를 제어할 때 사용하는 함수임. 이 함수는 스테핑 모터의 회전 속도를 설정하는 데 사용됨.
- delay(500) : 스테핑 모터가 동작 후 0.5초 동안 대기함. Delay() 함수는 일정 시간 동안 프로그램 실행을 일시정지시킴

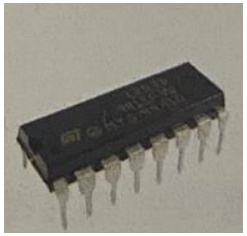
4. L298D DC 모터

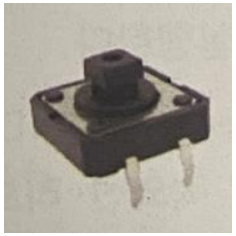
- 개요
  - 우노 R3를 이용하여 L293D 모터 드라이버와 DC모터 제어함
  - 프로그래밍 동작은 모터 드라이버인 L293D으로 DC모터의 반시계 방향 회전, 시계 방향 회전, 정지를 시킬 수 있는 예제임.

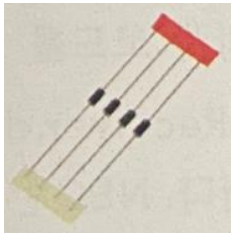
- 사용 디바이스 : 우노 R3(a), DC 모터(b), L298D(c), 스위치(d), 1N4001 다이오드(e), 저항 10k(f)


(a)


(b)


(c)


(d)

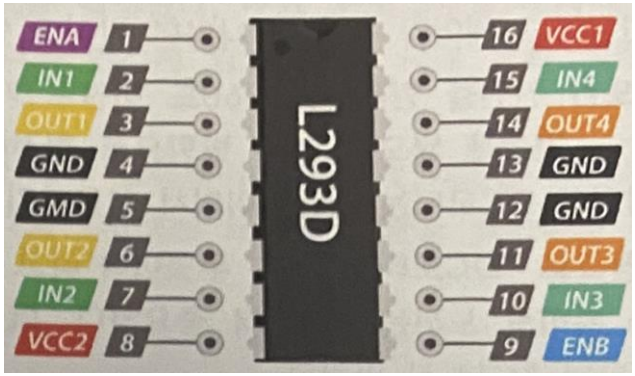

(e)


(f)

1N4001 다이오드 : 모터에 전류를 흘렸다가 끊으면 모터 코일에 역기전력(back EMF)이 발생. 모터에서 발생하는 역기전력(Back EMF)을 보호, 회로 손상 방지

4. L298D DC 모터

- L293D 모터드라이버 IC
  - <그림 2>처럼 L293D는 L298N 모터 드라이버와 더불어 대표적인 범용 DC모터 드라이버로 H-브릿지 회로가 내장된 소형 모터 드라이버임.
  - L293D 모터 드라이버 IC는 우노 R3와 DC모터 사이에서 인터페이스 되어 동작되며 동시에 2개의 모터 동작을 제어할 수 있음.
  - 또한 2개의 H 브릿지 회로는 저전력 DC모터를 제어하기에 적합하도록 설계되었음. 특히 L293D 모터 드라이버는 아래와 같은 기능을 제공함.
    - DC 모터 회전제어 정회전, 역회전
    - DC 모터 정지
    - DC 모터 속도조절(PWM 신호)



<그림 2. L293D 모터드라이버 IC>

4. L298D DC 모터

- DC모터 동작을 위한 입력 진리표
  - <표 1>에서는 DC모터 동작을 위한 입력 진리표를 보여줌. 입력 진리표에 따라 모터 제어 시 입력해야 할 신호를 아래에서 설명하고 있음.
  - IN1, IN2핀은 DC모터의 회전방향을 선택할 수 있는 핀으로 각각 하이 혹은 로우 신호를 입력 가능. 또한 이 핀들에 동시에 같은 신호를 입력할 경우에는 DC 모터에 동작을 정지시킬 수도 있음.
  - DC모터 속도 조절시에는 ENA 혹은 ENB에서 PWM 펄스를 입력해야 하며, 만일 2개의 DC모터 속도조절을 원할 경우에는 각각의 ENA, ENB핀에 PWM 펄스를 입력하여 조절함.

ENA, B 입력	IN1, 2 입력	IN1, 2 입력	모터 1,2 동작
하이 or PWM 펄스	하이(HIGH)	하이	정지
	하이	로우(LOW)	정방향 회전
	로우	하이	역방향 회전
	로우	로우	정지

<표 1> DC모터 동작을 위한 입력 진리표

4. L298D DC 모터

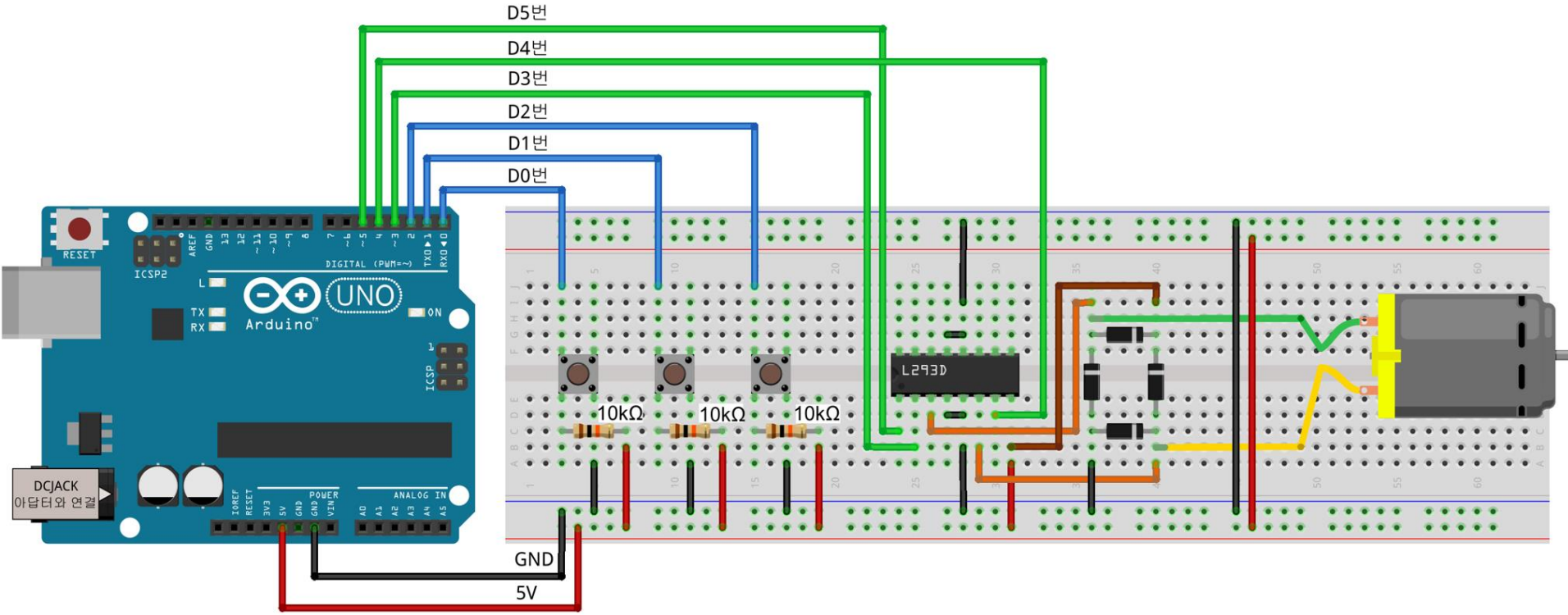
- DC 모터 드라이버가 왜 필요한가?
  - 우노 R3 출력 전압은 3.3~5V, 전류는 최대 25mA 수준임.
  - 이 전력으로는 DC 모터를 충분히 구동할 수 없어, 힘 없이 돌거나 정상 동작 불가능함.
  - 우노 R3로 직접 DC 모터를 구동하면 보드가 고장 나거나 반복적으로 리셋될 위험이 있음. 심하면 우노 R3와 연결된 PC에도 손상이 발생할 수 있음.
  - 따라서 우노 R3와 DC 모터 사이에는 L293D 같은 전용 모터 드라이버가 반드시 필요하고 모터 제어 시에는 항상 모터 드라이버를 사용해야 안전함.
- DC 모터 는 무엇인가?
  - DC 모터는 구조가 단순하고 전원을 연결하면 바로 회전하기 때문에 여러 모터 중 가장 다루기 쉬운 모터임.
  - 전원의 극성을 바꾸면 회전 방향이 변해 제어가 매우 직관적임. 예를 들어 전원을 + / – 방향으로 연결하면 시계 방향으로 회전하고, 극성을 반대로 하여 – / + 방향으로 연결하면 반시계 방향으로 회전함.

4. L298D DC 모터

- 모터 구현 회로도

우노 R3	스위치
디지털 0번	스위치1
디지털 1번	스위치2
디지털 2번	스위치3

우노 R3	L293D
디지털 3번	DIR1
디지털 4번	DIR2
디지털 5번	EN1
5V	+, VSS
GND	-, GND



4. L298D DC 모터

주요 프로그래밍 시나리오는 우노 R3에서 모터 L293D로 신호를 보내 DC모터의 반시계 방향 회전, 시계 방향 회전,정지를 시키는 동작을 구현한 예제이다. EN1은 DC모터 동작을 활성화 시키는 핀으로 하이 신호가 입력되어야 모터가 작동한다. 주로PWM을 넣어 모터의 속도 제어를 위해 사용되며 DIR1과 DIR2는 모터의 방향을 설정하는 핀이다.

```
int Switch1_Pin = 0; // 스위치1
int Switch2_Pin = 1; // 스위치2
int Switch3_Pin = 2; // 스위치3


int DIR1 = 3; // L293D DIR1
int DIR2 = 4; // L293D DIR2
int EN1 = 5; // L293D EN1


void Motor_ccw() { // 반시계 방향 회전
  digitalWrite(DIR1, HIGH); // DIR1 On
  digitalWrite(DIR2, LOW);  // DIR2 Off
  digitalWrite(EN1, HIGH);  // Motor Enable
}
void Motor_cw() // 시계 방향 회전
{
  digitalWrite(DIR1, LOW); // DIR1 Off
  digitalWrite(DIR2, HIGH);// DIR2 On
  digitalWrite(EN1, HIGH); // Motor Enable
}
void Motor_stop() // 회전 정지
{
  digitalWrite(DIR1, HIGH); // DIR1 On
  digitalWrite(DIR2, HIGH); // DIR2 On
  digitalWrite(EN1, LOW); // Motor Disable
}
```

```
void setup() // 초기화
{
  Serial.begin(9600);
  pinMode(Switch1_Pin, INPUT);  // 스위치 1번핀 입력포트 설정
  pinMode(Switch2_Pin, INPUT);  // 스위치 2번핀 입력포트 설정
  pinMode(Switch3_Pin, INPUT);  // 스위치 3번핀 입력포트 설정


  pinMode(DIR1, OUTPUT);    // DIR1 핀 출력포트 설정
  pinMode(DIR2, OUTPUT);    // DIR2 핀 출력포트 설정
  pinMode(EN1, OUTPUT);    // EN1 핀 출력포트 설정
}
/*
   스위치를 이용하여 DC모터에 정회전, 역회전, 정지하도록 한다.
*/
void loop() // 무한루프
{

  if (digitalRead(Switch1_Pin) == LOW)//1번째 스위치 클릭
  {
    Motor_ccw();
  }
  else if (digitalRead(Switch2_Pin) == LOW)//2번째 스위치 클릭
  {
    Motor_cw();
  }
  else if (digitalRead(Switch3_Pin) == LOW)//3번째 스위치 클릭
  {
    Motor_stop();
  }

}
```

Thank You